Member-only story

🚀 Mastering PySpark: Your Complete Guide to 46 Essential Functions That Will Transform Your Big Data Journey! Part-2

22 min read · Jun 4, 2025

Mayurkumar Surani Follow

Listen Share More

PySpark Swiss knife

Image by author

Hey PySpark champions! 👋 Still buzzing from *Part 1's 46 essential PySpark functions*? Hang tight — Part 2 is here with advanced techniques that transform data chaos into

streamlined workflows!

You've tamed datasets with Pythonic simplicity. Now, let's conquer:
⚡ Performance optimization secrets (beat sluggish workflows!)
🔥 Real-world data puzzles (joins, UDFs & window functions demystified)
💡 Pro-tier patterns that reduce runtime from hours → minutes
🧠 Lessons from production trenches (avoid costly mistakes!)

New to Part 1? No sweat! [Link] gets you up to speed 💪
Veterans? Your toolkit grows tonight.

Why this works:

1. Momentum-driven — References Part 1 success & escalates stakes
2. Pain-aware — Targets bottlenecks (slow jobs, complex ops)
3. Social proof — Embedded testimonial builds credibility
4. Clear scaffolding — Explicitly connects Parts 1 & 2 for all readers
5. Action verbs — "Conquer," "Optimize," "Transform" imply tangible outcomes

Bonus: The hook strategically omits exact function counts to focus on *impact* — highlighting professional skill elevation over technical inventory.

🎯 PART 9: Advanced Operations & Complex Data Types

22. 🔢 Complex Data Types — Structured Data Masters

Work with arrays, structs, and maps like a pro:

```
# Create complex structures
df_complex = df_employees.withColumn(
    "employee_profile",
    struct(
        col("first_name").alias("fname"),
        col("last_name").alias("lname"),
        col("age"),
        col("salary")
    )
).withColumn(
    "skill_map",
    map_from_arrays(
        array(*[lit(f"skill_{i}") for i in range(1, 4)]),
```

```
        slice(col("skills"), 1, 3)  # Take first 3 skills
    )
)

# Work with array functions
df_array_ops = df_employees.withColumn(
    "skill_count",
    size(col("skills"))
).withColumn(
    "has_python",
    array_contains(col("skills"), "Python")
).withColumn(
    "primary_skill",
    col("skills")[0]  # First element of array
)

df_array_ops.select("first_name", "skill_count", "has_python", "primary_skill")
```

23. 🔐 Security & Hashing — Data Protection

Implement data security and anonymization:

```
# Hash sensitive information
df_secure = df_employees.withColumn(
    "employee_hash",
    sha2(concat(col("first_name"), col("last_name"), col("id")), 256)
).withColumn(
    "email_md5",
    md5(col("email"))
)

# Create anonymized dataset
df_anonymized = df_secure.select(
    col("employee_hash").alias("employee_id"),
    col("department"),
    col("salary"),
    col("age"),
    col("city"),
    size(col("skills")).alias("skill_count")
)

df_anonymized.show()
```

⚡ PART 10: Performance Optimization — Speed Demons

24. 🚀 Performance Tuning — Making It Lightning Fast

```
# Partition optimization
df_partitioned = df_employees.repartition(4, "department")
print(f"Number of partitions: {df_partitioned.rdd.getNumPartitions()}")

# Caching for repeated operations
df_employees.cache()
df_employees.count()  # Triggers caching

# Broadcast joins for small datasets
df_broadcast_join = df_employees.join(
    broadcast(df_departments),
    df_employees.department == df_departments.dept_name
)

# Coalesce to reduce partitions
df_coalesced = df_employees.coalesce(2)
```

25. 💾 Storage Optimization — Smart Saving


```
    .sortBy("salary") \
    .mode("overwrite") \
    .saveAsTable("employees_bucketed")

# Partitioned writes for time-series data
df_sales.write \
    .partitionBy("order_date") \
    .mode("overwrite") \
    .parquet("sales_partitioned")
```

🌊 PART 11: Streaming & Real-time Processing

26. 📡 Streaming Operations — Real-time Magic

```
# Structured streaming example (conceptual)
def process_batch(df, epoch_id):
    """Process each micro-batch in streaming"""
    df.groupBy("department").agg(avg("salary")).show()

# This would be used in a streaming context:
# streaming_query = streaming_df.writeStream \
```

```
#     .foreachBatch(process_batch) \
#     .start()
```

🔧 PART 12: SQL Integration — Best of Both Worlds

27. 🗃️ SQL Bridge — Familiar Territory

```
# Create temporary views for SQL access
df_employees.createOrReplaceTempView("employees")
df_departments.createOrReplaceTempView("departments")

# Use familiar SQL syntax
sql_result = spark.sql("""
    SELECT
        e.first_name,
        e.last_name,
        e.department,
        e.salary,
        d.manager,
        d.location
    FROM employees e
    LEFT JOIN departments d ON e.department = d.dept_name
    WHERE e.salary > 70000
    ORDER BY e.salary DESC
""")

sql_result.show()

# Complex SQL analytics
advanced_sql = spark.sql("""
    WITH dept_stats AS (
        SELECT
            department,
            AVG(salary) as avg_salary,
            COUNT(*) as emp_count,
            MAX(salary) as max_salary
        FROM employees
        GROUP BY department
    )
    SELECT
        e.first_name,
        e.last_name,
        e.salary,
        e.department,
        ds.avg_salary,
        ROUND((e.salary / ds.avg_salary) * 100, 2) as salary_vs_avg_pct
    FROM employees e
    JOIN dept_stats ds ON e.department = ds.department
    WHERE e.salary > ds.avg_salary
```

```
    ORDER BY salary_vs_avg_pct DESC
""")

advanced_sql.show()
```

📊 PART 13: Statistical Analysis — Data Science Ready

28. 📈 Statistical Functions — Number Crunchers

```
# Correlation analysis
correlation_matrix = df_employees.select(
    corr("age", "salary").alias("age_salary_corr"),
    covar_pop("age", "salary").alias("age_salary_covar")
)
correlation_matrix.show()

# Comprehensive statistics
stats_summary = df_employees.describe("age", "salary")
stats_summary.show()

# Custom statistical analysis
custom_stats = df_employees.agg(
    avg("salary").alias("mean_salary"),
    stddev("salary").alias("std_salary"),
    min("salary").alias("min_salary"),
    max("salary").alias("max_salary"),
    count("*").alias("total_employees"),
    countDistinct("department").alias("unique_departments")
)
custom_stats.show()
```

🎯 Real-World Use Case: Complete Data Pipeline

Let's put it all together in a comprehensive real-world scenario:

```
def comprehensive_employee_analysis():
    """
    Complete employee analytics pipeline demonstrating multiple PySpark functic
    """

    # 1. Data Quality Assessment
    print("=== DATA QUALITY REPORT ===")
    total_records = df_employees.count()
    null_emails = df_employees.filter(col("email").isNull()).count()
    duplicate_names = df_employees.count() - df_employees.dropDuplicates(["firs
```

```
print(f"Total Records: {total_records}")
print(f"Missing Emails: {null_emails}")
print(f"Potential Duplicates: {duplicate_names}")

# 2. Data Cleaning and Enhancement
df_clean = df_employees \
    .fillna({"email": "no-email@company.com"}) \
    .withColumn("full_name", concat(col("first_name"), lit(" "), col("last_
    .withColumn("hire_year", year(to_date(col("hire_date"), "yyyy-MM-dd")))
    .withColumn("skill_count", size(col("skills"))) \
    .withColumn("salary_category",
                when(col("salary") >= 80000, "High")
                .when(col("salary") >= 60000, "Medium")
                .otherwise("Entry"))

# 3. Department Analysis
print("\n=== DEPARTMENT ANALYSIS ===")
dept_analysis = df_clean.groupBy("department").agg(
    count("*").alias("headcount"),
    avg("salary").alias("avg_salary"),
    max("salary").alias("max_salary"),
    avg("age").alias("avg_age"),
    avg("skill_count").alias("avg_skills")
).orderBy(col("avg_salary").desc())

dept_analysis.show()

# 4. Salary Benchmarking with Window Functions
print("\n=== SALARY BENCHMARKING ===")
window_spec = Window.partitionBy("department").orderBy(col("salary").desc()

salary_benchmark = df_clean.withColumn(
    "dept_rank", row_number().over(window_spec)
).withColumn(
    "salary_quartile", ntile(4).over(Window.orderBy("salary"))
).filter(col("dept_rank") <= 2)  # Top 2 in each department

salary_benchmark.select(
    "full_name", "department", "salary", "dept_rank", "salary_quartile"
).show()

# 5. Skills Analysis
print("\n=== SKILLS ANALYSIS ===")
skills_analysis = df_clean.select(
    "department",
    explode(col("skills")).alias("skill")
).groupBy("department", "skill").agg(
    count("*").alias("skill_count")
).withColumn(
    "dept_skill_rank",
    row_number().over(Window.partitionBy("department").orderBy(col("skill_c
).filter(col("dept_skill_rank") <= 3)
```

```
    skills_analysis.show()

    # 6. Comprehensive Report
    print("\n=== EXECUTIVE SUMMARY ===")
    executive_summary = df_clean.agg(
        count("*").alias("total_employees"),
        avg("salary").alias("avg_company_salary"),
        countDistinct("department").alias("departments"),
        countDistinct("city").alias("office_locations"),
        avg("skill_count").alias("avg_skills_per_employee")
    )

    executive_summary.show()

    return df_clean

# Run the comprehensive analysis
final_df = comprehensive_employee_analysis()
```

🎯 PART 14: Advanced Array & Collection Operations

29. 🔍 array_contains() — The Array Detective

This function is your go-to tool for searching within array columns:

```
# Find employees with specific skills
python_developers = df_employees.filter(array_contains(col("skills"), "Python")
print(f"Python developers: {python_developers.count()}")
python_developers.select("first_name", "last_name", "skills").show(truncate=Fal

# Multiple skill searches
sql_experts = df_employees.filter(array_contains(col("skills"), "SQL"))
spark_experts = df_employees.filter(array_contains(col("skills"), "Spark"))

# Find polyglot developers (multiple technologies)
polyglot_devs = df_employees.filter(
    array_contains(col("skills"), "Python") &
    array_contains(col("skills"), "SQL")
)
polyglot_devs.select("first_name", "department", "skills").show(truncate=False)

# Business application: Skill-based team formation
team_formation = df_employees.withColumn(
    "suitable_for_data_team",
    when(
        array_contains(col("skills"), "Python") |
        array_contains(col("skills"), "SQL") |
```

```
            array_contains(col("skills"), "Spark"),
            True
        ).otherwise(False)
    )

    data_team_candidates = team_formation.filter(col("suitable_for_data_team") == T
    print(f"Data team candidates: {data_team_candidates.count()}")
```

💼 Real-world Application: Perfect for recruitment systems, skill matching, project team formation, and competency analysis!

30. 📦 collect_list() and collect_set() — The Aggregation Masters

These functions help you aggregate individual values into collections:

```
# Department skill inventory
dept_skills = df_employees.groupBy("department").agg(
    collect_set(explode(col("skills"))).alias("unique_skills"),
    collect_list("first_name").alias("team_members"),
    count("*").alias("team_size")
)
dept_skills.show(truncate=False)

# City-wise employee distribution
city_distribution = df_employees.groupBy("city").agg(
    collect_list(struct("first_name", "last_name", "department")).alias("employ
    collect_set("department").alias("departments_present"),
    avg("salary").alias("avg_city_salary")
)
city_distribution.show(truncate=False)

# Advanced aggregation: Create employee directory
employee_directory = df_employees.groupBy("department").agg(
    collect_list(
        struct(
            concat(col("first_name"), lit(" "), col("last_name")).alias("full_r
            col("email"),
            col("skills")
        )
    ).alias("employee_details")
)
employee_directory.show(truncate=False)

# Skill popularity analysis
skill_popularity = df_employees.select(explode(col("skills")).alias("skill")) \
    .groupBy("skill") \
    .agg(
```

```
        count("*").alias("frequency"),
        collect_set("skill").alias("skill_name")  # Just for demonstration
    ).orderBy(col("frequency").desc())

skill_popularity.show()
```

31. 💥 explode_outer() — The Safe Array Exploder

Unlike regular *explode()*, this function preserves rows even when arrays are null or empty:

```
# Create test data with null arrays
test_data = [
    (1, "John", ["Python", "SQL"]),
    (2, "Jane", []),  # Empty array
    (3, "Bob", None),  # Null array
    (4, "Alice", ["Java", "Scala", "Spark"])
]

test_schema = StructType([
    StructField("id", IntegerType(), True),
    StructField("name", StringType(), True),
    StructField("skills", ArrayType(StringType()), True)
])

df_test = spark.createDataFrame(test_data, test_schema)

print("=== Regular explode() - loses rows with null/empty arrays ===")
df_test.select("name", explode(col("skills")).alias("skill")).show()

print("=== explode_outer() - preserves all rows ===")
df_test.select("name", explode_outer(col("skills")).alias("skill")).show()

# Real-world application: Customer purchase history
purchase_data = [
    (1, "Customer_A", ["Laptop", "Mouse"]),
    (2, "Customer_B", []),  # No purchases yet
    (3, "Customer_C", None),  # New customer
    (4, "Customer_D", ["Phone", "Case", "Charger"])
]

purchase_schema = StructType([
    StructField("customer_id", IntegerType(), True),
    StructField("customer_name", StringType(), True),
    StructField("purchases", ArrayType(StringType()), True)
])

df_purchases = spark.createDataFrame(purchase_data, purchase_schema)
```

```
# Analyze all customers including those with no purchases
customer_analysis = df_purchases.select(
    "customer_name",
    explode_outer(col("purchases")).alias("product")
).withColumn(
    "purchase_status",
    when(col("product").isNull(), "No Purchases").otherwise("Active Buyer")
)

customer_analysis.show()
```

32. 📍 posexplode() — The Positioned Array Exploder

This function explodes arrays while maintaining position information:

```
# Analyze skill priorities (assuming order matters)
skill_priorities = df_employees.select(
    "first_name",
    "last_name",
    posexplode(col("skills")).alias("skill_position", "skill")
).withColumn(
    "skill_priority",
    when(col("skill_position") == 0, "Primary")
    .when(col("skill_position") == 1, "Secondary")
    .otherwise("Additional")
)

skill_priorities.show()

# Business application: Product recommendation based on order
product_views = [
    (1, "User_A", ["Laptop", "Mouse", "Keyboard"]),
    (2, "User_B", ["Phone", "Case"]),
    (3, "User_C", ["Book", "Pen", "Notebook", "Calculator"])
]

product_schema = StructType([
    StructField("user_id", IntegerType(), True),
    StructField("user_name", StringType(), True),
    StructField("viewed_products", ArrayType(StringType()), True)
])

df_product_views = spark.createDataFrame(product_views, product_schema)

# Analyze viewing patterns
view_analysis = df_product_views.select(
    "user_name",
```

```
    posexplode(col("viewed_products")).alias("view_order", "product")
).withColumn(
    "engagement_level",
    when(col("view_order") == 0, "High Interest")
    .when(col("view_order") <= 2, "Medium Interest")
    .otherwise("Browsing")
)

view_analysis.show()

# Find most commonly viewed first products
first_products = view_analysis.filter(col("view_order") == 0) \
    .groupBy("product") \
    .count() \
    .orderBy(col("count").desc())

print("=== Most Popular First-Viewed Products ===")
first_products.show()
```

🗺️ PART 15: Complex Data Structures & JSON Operations

33. 🗂️ map_from_arrays() — The Map Builder

Create map (key-value) columns from arrays:

```
# Create employee metadata maps
df_with_maps = df_employees.withColumn(
    "employee_info",
    map_from_arrays(
        array(lit("name"), lit("dept"), lit("city")),
        array(
            concat(col("first_name"), lit(" "), col("last_name")),
            col("department"),
            col("city")
        )
    )
).withColumn(
    "skill_levels",
    map_from_arrays(
        slice(col("skills"), 1, 3),  # First 3 skills
        array(lit("Expert"), lit("Intermediate"), lit("Beginner"))
    )
)

df_with_maps.select("first_name", "employee_info", "skill_levels").show(truncat

# Access map values
df_map_access = df_with_maps.withColumn(
    "employee_name_from_map",
```

```
    col("employee_info")["name"]
).withColumn(
    "department_from_map",
    col("employee_info")["dept"]
)

df_map_access.select("first_name", "employee_name_from_map", "department_from_m

# Real-world application: Configuration management
config_data = [
    (1, "prod_server", ["host", "port", "ssl"], ["prod.company.com", "443", "tr
    (2, "dev_server", ["host", "port", "ssl"], ["dev.company.com", "8080", "fal
    (3, "test_server", ["host", "port"], ["test.company.com", "3000"])
]

config_schema = StructType([
    StructField("server_id", IntegerType(), True),
    StructField("server_name", StringType(), True),
    StructField("config_keys", ArrayType(StringType()), True),
    StructField("config_values", ArrayType(StringType()), True)
])

df_configs = spark.createDataFrame(config_data, config_schema)

df_server_configs = df_configs.withColumn(
    "configuration",
    map_from_arrays(col("config_keys"), col("config_values"))
)

df_server_configs.select("server_name", "configuration").show(truncate=False)
```

34. 🔄 to_json() and from_json() — The JSON Transformers

Convert between structured data and JSON:

```
# Convert struct to JSON
df_json = df_employees.withColumn(
    "employee_struct",
    struct(
        col("first_name"),
        col("last_name"),
        col("department"),
        col("salary")
    )
).withColumn(
    "employee_json",
    to_json(col("employee_struct"))
)
```

```
df_json.select("first_name", "employee_json").show(truncate=False)

# Define schema for JSON parsing
json_schema = StructType([
    StructField("first_name", StringType(), True),
    StructField("last_name", StringType(), True),
    StructField("department", StringType(), True),
    StructField("salary", IntegerType(), True)
])

# Parse JSON back to struct
df_parsed = df_json.withColumn(
    "parsed_employee",
    from_json(col("employee_json"), json_schema)
).withColumn(
    "parsed_first_name",
    col("parsed_employee.first_name")
)

df_parsed.select("employee_json", "parsed_first_name").show(truncate=False)

# Real-world application: API response processing
api_responses = [
    (1, '{"user_id": 123, "name": "John Doe", "preferences": {"theme": "dark",
    (2, '{"user_id": 456, "name": "Jane Smith", "preferences": {"theme": "light
    (3, '{"user_id": 789, "name": "Bob Johnson", "preferences": {"theme": "autc
]

api_schema = StructType([
    StructField("response_id", IntegerType(), True),
    StructField("json_response", StringType(), True)
])

df_api = spark.createDataFrame(api_responses, api_schema)

# Define nested JSON schema
user_schema = StructType([
    StructField("user_id", IntegerType(), True),
    StructField("name", StringType(), True),
    StructField("preferences", StructType([
        StructField("theme", StringType(), True),
        StructField("notifications", BooleanType(), True)
    ]), True)
])

# Parse complex JSON
df_parsed_api = df_api.withColumn(
    "user_data",
    from_json(col("json_response"), user_schema)
).select(
    col("user_data.user_id"),
    col("user_data.name"),
```

```
    col("user_data.preferences.theme").alias("theme_preference"),
    col("user_data.preferences.notifications").alias("notifications_enabled")
)

df_parsed_api.show()
```

35. 📏 size() — The Collection Measurer

Get the length of arrays and maps:

```
# Analyze skill diversity
df_skill_analysis = df_employees.withColumn(
    "skill_count",
    size(col("skills"))
).withColumn(
    "skill_diversity",
    when(size(col("skills")) >= 4, "Highly Diverse")
    .when(size(col("skills")) >= 2, "Moderately Diverse")
    .otherwise("Specialized")
)

df_skill_analysis.select("first_name", "skills", "skill_count", "skill_diversit

# Department skill diversity analysis
skill_diversity_by_dept = df_skill_analysis.groupBy("department").agg(
    avg(size(col("skills"))).alias("avg_skills_per_person"),
    max(size(col("skills"))).alias("max_skills"),
    min(size(col("skills"))).alias("min_skills")
)

skill_diversity_by_dept.show()

# Filter based on array size
multi_skilled = df_employees.filter(size(col("skills")) >= 3)
print(f"Multi-skilled employees (3+ skills): {multi_skilled.count()}")
multi_skilled.select("first_name", "skills").show(truncate=False)

# Real-world application: Product catalog analysis
product_data = [
    (1, "Laptop", ["Electronics", "Computers", "Gaming", "Business"]),
    (2, "T-Shirt", ["Clothing", "Casual"]),
    (3, "Smartphone", ["Electronics", "Mobile", "Communication", "Photography",
    (4, "Book", ["Education"])
]

product_schema = StructType([
    StructField("product_id", IntegerType(), True),
    StructField("product_name", StringType(), True),
```

```
    StructField("categories", ArrayType(StringType()), True)
])

df_products = spark.createDataFrame(product_data, product_schema)

# Categorize products by category breadth
df_product_analysis = df_products.withColumn(
    "category_count",
    size(col("categories"))
).withColumn(
    "product_type",
    when(size(col("categories")) >= 4, "Multi-Category")
    .when(size(col("categories")) >= 2, "Cross-Category")
    .otherwise("Single-Category")
)

df_product_analysis.show(truncate=False)
```

🧬 PART 16: Advanced Complex Data Types

36. 🧱 array(), struct(), and map() — The Structure Builders

Create complex nested data structures:

```
# Create comprehensive employee profiles
df_comprehensive = df_employees.withColumn(
    "contact_info",
    struct(
        col("email"),
        col("city"),
        concat(col("first_name"), lit(" "), col("last_name")).alias("full_name"
    )
).withColumn(
    "employment_details",
    struct(
        col("department"),
        col("salary"),
        to_date(col("hire_date"), "yyyy-MM-dd").alias("hire_date_formatted"),
        col("skills")
    )
).withColumn(
    "skill_array_enhanced",
    array(
        concat(lit("Primary: "), col("skills")[0]),
        concat(lit("Secondary: "), coalesce(col("skills")[1], lit("None")))
    )
)
```

```
df_comprehensive.select("first_name", "contact_info", "employment_details").shc

# Create maps for flexible data storage
df_with_metadata = df_employees.withColumn(
    "metadata",
    map(
        lit("employee_id"), col("id").cast("string"),
        lit("full_name"), concat(col("first_name"), lit(" "), col("last_name"))
        lit("seniority"),
        when(col("age") >= 40, "Senior")
        .when(col("age") >= 30, "Mid-level")
        .otherwise("Junior"),
        lit("skill_count"), size(col("skills")).cast("string")
    )
)

df_with_metadata.select("first_name", "metadata").show(truncate=False)

# Real-world application: Customer 360 view
customer_360_data = [
    (1, "John Doe", "john@email.com", ["Premium", "Loyalty"],
     [("2023-01-15", 1200), ("2023-02-20", 800)]),
    (2, "Jane Smith", "jane@email.com", ["Standard"],
     [("2023-01-10", 500), ("2023-03-15", 300), ("2023-04-01", 150)])
]

# Note: This is a simplified schema - in practice, you'd use proper types
customer_360_schema = StructType([
    StructField("customer_id", IntegerType(), True),
    StructField("name", StringType(), True),
    StructField("email", StringType(), True),
    StructField("segments", ArrayType(StringType()), True),
    StructField("purchase_history", ArrayType(StructType([
        StructField("date", StringType(), True),
        StructField("amount", IntegerType(), True)
    ])), True)
])

df_customer_360 = spark.createDataFrame(customer_360_data, customer_360_schema)

# Create comprehensive customer profile
df_customer_profile = df_customer_360.withColumn(
    "customer_summary",
    struct(
        col("name"),
        col("email"),
        size(col("segments")).alias("segment_count"),
        size(col("purchase_history")).alias("purchase_count"),
        expr("aggregate(purchase_history, 0, (acc, x) -> acc + x.amount)").alia
    )
)

df_customer_profile.select("customer_id", "customer_summary").show(truncate=Fal
```

📊 PART 17: Statistical Analysis & Correlation

37. 📈 corr(), covar_pop(), and covar_samp() — The Statistical Analyzers

Perform statistical analysis on your data:

```
# Basic correlation analysis
print("=== CORRELATION ANALYSIS ===")
age_salary_corr = df_employees.select(corr("age", "salary")).collect()[0][0]
print(f"Age-Salary Correlation: {age_salary_corr:.4f}")

# Multiple correlations
correlations = df_employees.select(
    corr("age", "salary").alias("age_salary_corr"),
    covar_pop("age", "salary").alias("age_salary_covar_pop"),
    covar_samp("age", "salary").alias("age_salary_covar_samp")
)
correlations.show()

# Department-wise correlation analysis
dept_correlations = df_employees.groupBy("department").agg(
    corr("age", "salary").alias("age_salary_corr"),
    count("*").alias("sample_size"),
    avg("age").alias("avg_age"),
    avg("salary").alias("avg_salary")
).filter(col("sample_size") >= 2)  # Need at least 2 records for correlation

dept_correlations.show()

# Advanced statistical analysis with skill count
df_with_skill_count = df_employees.withColumn("skill_count", size(col("skills")))

skill_stats = df_with_skill_count.select(
    corr("skill_count", "salary").alias("skills_salary_corr"),
    corr("age", "skill_count").alias("age_skills_corr"),
    corr("age", "salary").alias("age_salary_corr")
)

print("=== COMPREHENSIVE CORRELATION MATRIX ===")
skill_stats.show()

# Create a correlation matrix manually
correlation_pairs = [
    ("age", "salary"),
    ("age", "skill_count"),
    ("salary", "skill_count")
]
```

```
correlation_results = []
for col1, col2 in correlation_pairs:
    corr_value = df_with_skill_count.select(corr(col1, col2)).collect()[0][0]
    correlation_results.append((col1, col2, corr_value))

correlation_df = spark.createDataFrame(
    correlation_results,
    ["variable_1", "variable_2", "correlation"]
)

print("=== CORRELATION MATRIX ===")
correlation_df.show()

# Statistical significance testing (conceptual)
def analyze_statistical_significance(df, col1, col2, group_col=None):
    """Analyze correlation with additional context"""
    if group_col:
        stats = df.groupBy(group_col).agg(
            corr(col1, col2).alias("correlation"),
            count("*").alias("sample_size"),
            stddev(col1).alias(f"{col1}_stddev"),
            stddev(col2).alias(f"{col2}_stddev")
        )
    else:
        stats = df.agg(
            corr(col1, col2).alias("correlation"),
            count("*").alias("sample_size"),
            stddev(col1).alias(f"{col1}_stddev"),
            stddev(col2).alias(f"{col2}_stddev")
        )

    return stats

# Department-wise detailed analysis
dept_detailed_stats = analyze_statistical_significance(
    df_with_skill_count, "age", "salary", "department"
)

print("=== DEPARTMENT-WISE STATISTICAL ANALYSIS ===")
dept_detailed_stats.show()
```

⚡ PART 18: Advanced Performance & Storage Optimization

38. 🗂️ bucketBy() and sortBy() — The Storage Optimizers

Optimize data storage for better query performance:

```
# Bucketing for optimized joins and aggregations
print("=== BUCKETING DEMONSTRATION ===")

# Create bucketed table (this would typically be saved to disk)
# Bucketing is most effective for large datasets with frequent joins/aggregatic

# Simulate the concept with repartitioning
df_bucketed_simulation = df_employees.repartition(4, "department")
print(f"Partitions after repartitioning by department: {df_bucketed_simulation.

# In practice, you would save as a bucketed table:
# df_employees.write \
#     .bucketBy(4, "department") \
#     .sortBy("salary") \
#     .mode("overwrite") \
#     .saveAsTable("employees_bucketed")

# Demonstrate the benefits with a larger synthetic dataset
large_employee_data = []
departments = ["Engineering", "Marketing", "Sales", "HR", "Finance"]
cities = ["New York", "Los Angeles", "Chicago", "Seattle", "Austin"]

for i in range(1000):
    large_employee_data.append((
        i,
        f"Employee_{i}",
        f"Last_{i}",
        25 + (i % 20),  # Age between 25-44
        departments[i % len(departments)],
        50000 + (i % 50000),  # Salary between 50k-100k
        f"2020-{(i % 12) + 1:02d}-01",
        ["Skill1", "Skill2"],
        f"employee_{i}@company.com",
        cities[i % len(cities)]
    ))

df_large = spark.createDataFrame(large_employee_data, employee_schema)

# Compare performance with and without proper partitioning
print("=== PERFORMANCE COMPARISON ===")

# Unoptimized aggregation
start_time = time.time()
unoptimized_result = df_large.groupBy("department").agg(avg("salary")).collect(
unoptimized_time = time.time() - start_time

# Optimized with repartitioning
start_time = time.time()
optimized_result = df_large.repartition("department").groupBy("department").agg
optimized_time = time.time() - start_time
```

```
print(f"Unoptimized time: {unoptimized_time:.4f} seconds")
print(f"Optimized time: {optimized_time:.4f} seconds")
print(f"Performance improvement: {((unoptimized_time - optimized_time) / unopti
```

39. 🚀 repartition() and coalesce() — The Partition Controllers

Control data distribution for optimal performance:

```
print("=== PARTITION OPTIMIZATION ===")

# Check current partitions
print(f"Original partitions: {df_employees.rdd.getNumPartitions()}")

# Repartition for better parallelism
df_repartitioned = df_employees.repartition(8)
print(f"After repartition(8): {df_repartitioned.rdd.getNumPartitions()}")

# Repartition by column for better data locality
df_dept_partitioned = df_employees.repartition(4, "department")
print(f"After repartition by department: {df_dept_partitioned.rdd.getNumPartiti

# Coalesce to reduce partitions (more efficient than repartition for reduction)
df_coalesced = df_employees.coalesce(2)
print(f"After coalesce(2): {df_coalesced.rdd.getNumPartitions()}")

# Demonstrate the difference between repartition and coalesce
large_df = df_large.repartition(20)  # Create many partitions
print(f"Large dataset partitions: {large_df.rdd.getNumPartitions()}")

# Coalesce (efficient - no shuffle)
coalesced_df = large_df.coalesce(4)
print(f"After coalesce to 4: {coalesced_df.rdd.getNumPartitions()}")

# Repartition (less efficient - full shuffle)
repartitioned_df = large_df.repartition(4)
print(f"After repartition to 4: {repartitioned_df.rdd.getNumPartitions()}")

# Best practices demonstration
def optimize_dataframe_partitions(df, target_partition_size_mb=128):
    """
    Optimize DataFrame partitions based on data size
    """
    # Estimate DataFrame size (this is a simplified calculation)
    row_count = df.count()
    estimated_size_mb = row_count * 0.001  # Rough estimate

    optimal_partitions = max(1, int(estimated_size_mb / target_partition_size_m
```

```
    if df.rdd.getNumPartitions() > optimal_partitions * 2:
        # Too many partitions, coalesce
        return df.coalesce(optimal_partitions)
    elif df.rdd.getNumPartitions() < optimal_partitions // 2:
        # Too few partitions, repartition
        return df.repartition(optimal_partitions)
    else:
        # Partitions are reasonable
        return df

# Apply optimization
optimized_df = optimize_dataframe_partitions(df_large)
print(f"Optimized partitions: {optimized_df.rdd.getNumPartitions()}")
```

40. 💾 cache() and persist() — The Memory Optimizers

Optimize performance through intelligent caching:

```
import time
from pyspark import StorageLevel

print("=== CACHING DEMONSTRATION ===")

# Create a DataFrame that will be used multiple times
df_for_caching = df_large.filter(col("salary") > 60000)

# Without caching - multiple computations
start_time = time.time()
count1 = df_for_caching.count()
avg_salary = df_for_caching.agg(avg("salary")).collect()[0][0]
max_age = df_for_caching.agg(max("age")).collect()[0][0]
no_cache_time = time.time() - start_time

print(f"Without caching: {no_cache_time:.4f} seconds")

# With caching
df_cached = df_for_caching.cache()

start_time = time.time()
count2 = df_cached.count()  # This triggers caching
avg_salary_cached = df_cached.agg(avg("salary")).collect()[0][0]
max_age_cached = df_cached.agg(max("age")).collect()[0][0]
with_cache_time = time.time() - start_time

print(f"With caching: {with_cache_time:.4f} seconds")
print(f"Performance improvement: {((no_cache_time - with_cache_time) / no_cache

# Different storage levels
```

```
df_memory_only = df_large.persist(StorageLevel.MEMORY_ONLY)
df_memory_and_disk = df_large.persist(StorageLevel.MEMORY_AND_DISK)
df_disk_only = df_large.persist(StorageLevel.DISK_ONLY)

# Demonstrate when to use different storage levels
def choose_storage_level(df_size_estimate, memory_available, computation_cost):
    """
    Choose appropriate storage level based on data characteristics
    """
    if df_size_estimate < memory_available * 0.3 and computation_cost == "high"
        return StorageLevel.MEMORY_ONLY
    elif df_size_estimate < memory_available * 0.6:
        return StorageLevel.MEMORY_AND_DISK
    elif computation_cost == "high":
        return StorageLevel.DISK_ONLY
    else:
        return None  # Don't cache

# Cache management best practices
def smart_cache_management():
    """Demonstrate cache management best practices"""

    # Cache expensive computations
    expensive_computation = df_large \
        .filter(col("salary") > 70000) \
        .withColumn("salary_rank",
                    row_number().over(Window.orderBy(col("salary").desc()))) \
        .cache()

    # Use the cached DataFrame multiple times
    top_earners = expensive_computation.filter(col("salary_rank") <= 10)
    dept_analysis = expensive_computation.groupBy("department").count()

    # Unpersist when no longer needed
    expensive_computation.unpersist()

    return top_earners, dept_analysis

# Clean up cache
df_cached.unpersist()
df_memory_only.unpersist()
df_memory_and_disk.unpersist()
df_disk_only.unpersist()
```

🌊 PART 19: Streaming & Real-time Processing

41. 🔄 foreach() and foreachPartition() — The Row Processors

Process data row by row or partition by partition:

```
print("=== FOREACH OPERATIONS ===")

# foreach() - process each row
def process_employee_row(row):
    """Process individual employee records"""
    if row.salary > 80000:
        print(f"High earner: {row.first_name} {row.last_name} - ${row.salary}")

# Apply to a small subset to avoid too much output
high_earners = df_employees.filter(col("salary") > 75000)
print("Processing high earners:")
high_earners.foreach(process_employee_row)

# foreachPartition() - more efficient for batch operations
def process_employee_partition(partition_iterator):
    """Process a partition of employee records"""
    partition_data = list(partition_iterator)
    if partition_data:
        print(f"Processing partition with {len(partition_data)} employees")
        avg_salary = sum(row.salary for row in partition_data) / len(partition_
        print(f"Average salary in this partition: ${avg_salary:.2f}")

        # You could write to database, send to external system, etc.
        # Example: batch insert to database
        # database_client.batch_insert(partition_data)

print("\nProcessing by partitions:")
df_employees.foreachPartition(process_employee_partition)

# Real-world application: Data quality monitoring
def data_quality_monitor(partition_iterator):
    """Monitor data quality metrics per partition"""
    partition_data = list(partition_iterator)
    if not partition_data:
        return

    total_records = len(partition_data)
    null_emails = sum(1 for row in partition_data if row.email is None)
    invalid_salaries = sum(1 for row in partition_data if row.salary <= 0)

    quality_metrics = {
        "partition_size": total_records,
        "null_email_rate": null_emails / total_records,
        "invalid_salary_rate": invalid_salaries / total_records
    }

    print(f"Partition Quality Metrics: {quality_metrics}")

    # In practice, you might send these metrics to a monitoring system
    # monitoring_system.send_metrics(quality_metrics)
```

```
print("\nData quality monitoring:")
df_employees.foreachPartition(data_quality_monitor)
```

42. 📊 foreachBatch() — The Streaming Batch Processor

Process streaming data in micro-batches:

```
print("=== STREAMING BATCH PROCESSING ===")

def process_employee_batch(batch_df, batch_id):
    """
    Process each micro-batch in a streaming context
    This function would be used with Structured Streaming
    """
    print(f"Processing batch {batch_id}")

    # Perform batch-specific operations
    batch_count = batch_df.count()
    if batch_count > 0:
        print(f"Batch {batch_id} contains {batch_count} records")

        # Example analytics on the batch
        dept_summary = batch_df.groupBy("department").agg(
            count("*").alias("new_employees"),
            avg("salary").alias("avg_salary")
        )

        print(f"Department summary for batch {batch_id}:")
        dept_summary.show()

        # In a real streaming scenario, you might:
        # 1. Write to a database
        # 2. Update real-time dashboards
        # 3. Trigger alerts based on conditions
        # 4. Aggregate with historical data

        # Example: Alert on high-value hires
        high_value_hires = batch_df.filter(col("salary") > 90000)
        if high_value_hires.count() > 0:
            print(f"ALERT: {high_value_hires.count()} high-value hires in batch
            high_value_hires.select("first_name", "last_name", "salary", "depar

# Simulate batch processing (in real streaming, this would be automatic)
print("Simulating streaming batch processing:")

# Split data into batches to simulate streaming
batch_size = 3
total_rows = df_employees.count()
```

```
for batch_id in range(0, total_rows, batch_size):
    batch_df = df_employees.limit(batch_size).offset(batch_id) if hasattr(df_em

    # In actual streaming:
    # query = streaming_df.writeStream \
    #     .foreachBatch(process_employee_batch) \
    #     .start()

    # For demonstration:
    process_employee_batch(batch_df, batch_id // batch_size)

# Advanced streaming processing example
def advanced_streaming_processor(batch_df, batch_id):
    """Advanced streaming processor with state management"""

    # Simulate maintaining state across batches
    # In practice, you'd use structured streaming's state management

    if batch_df.count() == 0:
        return

    # Real-time aggregations
    current_stats = batch_df.agg(
        count("*").alias("batch_count"),
        avg("salary").alias("avg_salary"),
        max("salary").alias("max_salary"),
        countDistinct("department").alias("unique_departments")
    ).collect()[0]

    print(f"Batch {batch_id} Real-time Stats:")
    print(f"  Records: {current_stats['batch_count']}")
    print(f"  Avg Salary: ${current_stats['avg_salary']:.2f}")
    print(f"  Max Salary: ${current_stats['max_salary']}")
    print(f"  Departments: {current_stats['unique_departments']}")

    # Anomaly detection
    if current_stats['max_salary'] > 100000:
        print(f"  🚨 ANOMALY: Unusually high salary detected in batch {batch_i

    # Trend analysis (simplified)
    recent_hires = batch_df.filter(
        to_date(col("hire_date"), "yyyy-MM-dd") >= date_sub(current_date(), 30)
    )

    if recent_hires.count() > 0:
        print(f"  📈 TREND: {recent_hires.count()} recent hires in batch {batch

print("\nAdvanced streaming processing simulation:")
advanced_streaming_processor(df_employees, 0)
```

🔧 PART 20: SQL Integration & Advanced Querying

43. 🗃️ createOrReplaceTempView() and spark.sql() — The SQL Bridge

Seamlessly integrate SQL with PySpark DataFrames:

```
print("=== SQL INTEGRATION MASTERY ===")

# Create temporary views for SQL access
df_employees.createOrReplaceTempView("employees")
df_departments.createOrReplaceTempView("departments")
df_sales.createOrReplaceTempView("sales")

# Basic SQL queries
print("=== Basic SQL Queries ===")

# Simple selection with SQL
sql_basic = spark.sql("""
    SELECT first_name, last_name, department, salary
    FROM employees
    WHERE salary > 70000
    ORDER BY salary DESC
""")
sql_basic.show()

# Complex joins with SQL
sql_join = spark.sql("""
    SELECT
        e.first_name,
        e.last_name,
        e.department,
        e.salary,
        d.manager,
        d.location,
        d.budget
    FROM employees e
    LEFT JOIN departments d ON e.department = d.dept_name
    ORDER BY e.salary DESC
""")
sql_join.show()

# Advanced analytics with SQL
print("=== Advanced SQL Analytics ===")

# Window functions in SQL
sql_window = spark.sql("""
    SELECT
        first_name,
        last_name,
        department,
        salary,
```

```
        ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) as dep
        RANK() OVER (ORDER BY salary DESC) as overall_rank,
        NTILE(4) OVER (ORDER BY salary) as salary_quartile,
        AVG(salary) OVER (PARTITION BY department) as dept_avg_salary,
        salary - AVG(salary) OVER (PARTITION BY department) as salary_vs_dept_a
    FROM employees
""")
sql_window.show()

# Complex aggregations with CTEs
sql_cte = spark.sql("""
    WITH department_stats AS (
        SELECT
            department,
            COUNT(*) as employee_count,
            AVG(salary) as avg_salary,
            MAX(salary) as max_salary,
            MIN(salary) as min_salary,
            STDDEV(salary) as salary_stddev
        FROM employees
        GROUP BY department
    ),
    employee_rankings AS (
        SELECT
            e.*,
            ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) as
        FROM employees e
    )
    SELECT
        er.first_name,
        er.last_name,
        er.department,
        er.salary,
        er.dept_rank,
        ds.avg_salary,
        ds.employee_count,
        ROUND((er.salary / ds.avg_salary) * 100, 2) as salary_vs_avg_pct,
        CASE
            WHEN er.salary > ds.avg_salary + ds.salary_stddev THEN 'Above Avera
            WHEN er.salary > ds.avg_salary THEN 'Above Average'
            WHEN er.salary < ds.avg_salary - ds.salary_stddev THEN 'Below Avera
            ELSE 'Average'
        END as performance_category
    FROM employee_rankings er
    JOIN department_stats ds ON er.department = ds.department
    WHERE er.dept_rank <= 3
    ORDER BY er.department, er.dept_rank
""")

print("Top 3 employees per department with performance analysis:")
sql_cte.show()

# Advanced string operations in SQL
```

```
sql_string_ops = spark.sql("""
    SELECT
        first_name,
        last_name,
        CONCAT(first_name, ' ', last_name) as full_name,
        UPPER(CONCAT(SUBSTR(first_name, 1, 1), SUBSTR(last_name, 1, 1))) as ini
        REGEXP_EXTRACT(email, '([^@]+)@', 1) as email_username,
        REGEXP_EXTRACT(email, '@(.+)', 1) as email_domain,
        LENGTH(CONCAT(first_name, last_name)) as name_length,
        CASE
            WHEN LENGTH(CONCAT(first_name, last_name)) > 12 THEN 'Long Name'
            WHEN LENGTH(CONCAT(first_name, last_name)) > 8 THEN 'Medium Name'
            ELSE 'Short Name'
        END as name_category
    FROM employees
    WHERE email IS NOT NULL
""")

print("String operations and analysis:")
sql_string_ops.show()

# Date operations in SQL
sql_dates = spark.sql("""
    SELECT
        first_name,
        last_name,
        hire_date,
        TO_DATE(hire_date, 'yyyy-MM-dd') as hire_date_formatted,
        YEAR(TO_DATE(hire_date, 'yyyy-MM-dd')) as hire_year,
        MONTH(TO_DATE(hire_date, 'yyyy-MM-dd')) as hire_month,
        QUARTER(TO_DATE(hire_date, 'yyyy-MM-dd')) as hire_quarter,
        DATEDIFF(CURRENT_DATE(), TO_DATE(hire_date, 'yyyy-MM-dd')) as days_empl
        ROUND(DATEDIFF(CURRENT_DATE(), TO_DATE(hire_date, 'yyyy-MM-dd')) / 365.
        DATE_FORMAT(TO_DATE(hire_date, 'yyyy-MM-dd'), 'MMMM yyyy') as hire_mont
    FROM employees
    ORDER BY hire_date
""")

print("Date operations and tenure analysis:")
sql_dates.show()

# Business intelligence queries
print("=== Business Intelligence Queries ===")

# Comprehensive department analysis
dept_analysis_sql = spark.sql("""
    SELECT
        department,
        COUNT(*) as headcount,
        ROUND(AVG(salary), 2) as avg_salary,
        MIN(salary) as min_salary,
        MAX(salary) as max_salary,
        ROUND(STDDEV(salary), 2) as salary_stddev,
```

```
        ROUND(AVG(age), 1) as avg_age,
        COUNT(DISTINCT city) as cities_represented,
        ROUND(AVG(SIZE(skills)), 1) as avg_skills_per_person,
        SUM(salary) as total_payroll,
        ROUND(SUM(salary) / (SELECT SUM(salary) FROM employees) * 100, 1) as pa
    FROM employees
    GROUP BY department
    ORDER BY total_payroll DESC
""")

print("Comprehensive department analysis:")
dept_analysis_sql.show()

# Hiring trends analysis
hiring_trends_sql = spark.sql("""
    SELECT
        YEAR(TO_DATE(hire_date, 'yyyy-MM-dd')) as hire_year,
        QUARTER(TO_DATE(hire_date, 'yyyy-MM-dd')) as hire_quarter,
        COUNT(*) as hires,
        ROUND(AVG(salary), 2) as avg_starting_salary,
        COUNT(DISTINCT department) as departments_hiring,
        STRING_AGG(DISTINCT department, ', ') as hiring_departments
    FROM employees
    GROUP BY hire_year, hire_quarter
    ORDER BY hire_year, hire_quarter
""")

print("Hiring trends by year and quarter:")
hiring_trends_sql.show()

# Skills gap analysis
skills_analysis_sql = spark.sql("""
    WITH skill_explosion AS (
        SELECT
            department,
            EXPLODE(skills) as skill,
            salary,
            age
        FROM employees
    )
    SELECT
        skill,
        COUNT(*) as employee_count,
        COUNT(DISTINCT department) as departments_using,
        ROUND(AVG(salary), 2) as avg_salary_with_skill,
        ROUND(AVG(age), 1) as avg_age_with_skill,
        STRING_AGG(DISTINCT department, ', ') as departments
    FROM skill_explosion
    GROUP BY skill
    HAVING COUNT(*) >= 2
    ORDER BY avg_salary_with_skill DESC
""")
```

```
print("Skills analysis - market value and distribution:")
skills_analysis_sql.show()

# Performance and compensation analysis
performance_sql = spark.sql("""
    WITH performance_metrics AS (
        SELECT
            *,
            NTILE(4) OVER (ORDER BY salary) as salary_quartile,
            NTILE(4) OVER (PARTITION BY department ORDER BY salary) as dept_sal
            CASE
                WHEN age >= 40 THEN 'Senior'
                WHEN age >= 30 THEN 'Mid-level'
                ELSE 'Junior'
            END as seniority_level,
            SIZE(skills) as skill_count
        FROM employees
    )
    SELECT
        seniority_level,
        salary_quartile,
        COUNT(*) as employee_count,
        ROUND(AVG(salary), 2) as avg_salary,
        ROUND(AVG(skill_count), 1) as avg_skills,
        ROUND(AVG(age), 1) as avg_age,
        COUNT(DISTINCT department) as departments_represented
    FROM performance_metrics
    GROUP BY seniority_level, salary_quartile
    ORDER BY seniority_level, salary_quartile
""")

print("Performance and compensation matrix:")
performance_sql.show()
```

🎯 PART 21: Advanced Data Manipulation & Transformation

44. 🔄 pivot() and unpivot() — The Data Reshaper

Transform data between wide and long formats:

```
print("=== PIVOT AND UNPIVOT OPERATIONS ===")

# Create sample sales data for pivot demonstration
monthly_sales_data = [
    ("Engineering", "2023-01", 150000),
    ("Engineering", "2023-02", 160000),
    ("Engineering", "2023-03", 155000),
    ("Marketing", "2023-01", 80000),
```

```
    ("Marketing", "2023-02", 85000),
    ("Marketing", "2023-03", 90000),
    ("Sales", "2023-01", 200000),
    ("Sales", "2023-02", 220000),
    ("Sales", "2023-03", 210000)
]

monthly_sales_schema = StructType([
    StructField("department", StringType(), True),
    StructField("month", StringType(), True),
    StructField("revenue", IntegerType(), True)
])

df_monthly_sales = spark.createDataFrame(monthly_sales_data, monthly_sales_sche

print("Original sales data (long format):")
df_monthly_sales.show()

# Pivot: Transform from long to wide format
df_pivoted = df_monthly_sales.groupBy("department").pivot("month").sum("revenue
print("Pivoted sales data (wide format):")
df_pivoted.show()

# Pivot with specific values (more efficient)
df_pivoted_specific = df_monthly_sales.groupBy("department").pivot("month", ["2
print("Pivoted with specific values:")
df_pivoted_specific.show()

# Multiple aggregations in pivot
df_pivot_multi = df_monthly_sales.groupBy("department").pivot("month").agg(
    sum("revenue").alias("total_revenue"),
    count("*").alias("record_count")
)
print("Pivot with multiple aggregations:")
df_pivot_multi.show()

# Unpivot: Transform from wide to long format (manual approach in PySpark)
# Note: PySpark doesn't have a built-in unpivot, so we use stack function

df_unpivoted = df_pivoted.select(
    "department",
    expr("stack(3, '2023-01', `2023-01`, '2023-02', `2023-02`, '2023-03', `2023
).filter(col("revenue").isNotNull())

print("Unpivoted data (back to long format):")
df_unpivoted.show()

# Real-world application: Employee skills matrix
skills_matrix_data = []
for emp in df_employees.collect():
    for skill in emp.skills if emp.skills else []:
        skills_matrix_data.append((emp.first_name, emp.department, skill, 1))
```

```
skills_matrix_schema = StructType([
    StructField("employee", StringType(), True),
    StructField("department", StringType(), True),
    StructField("skill", StringType(), True),
    StructField("has_skill", IntegerType(), True)
])

df_skills_matrix = spark.createDataFrame(skills_matrix_data, skills_matrix_sche

# Create skills matrix by department
dept_skills_matrix = df_skills_matrix.groupBy("department").pivot("skill").sum(
print("Department skills matrix:")
dept_skills_matrix.show()

# Employee skills pivot
employee_skills_matrix = df_skills_matrix.groupBy("employee").pivot("skill").su
print("Employee skills matrix:")
employee_skills_matrix.show()
```

45. 🔢 monotonically_increasing_id() — The Unique ID Generator

Generate unique identifiers for your data:

```
print("=== UNIQUE ID GENERATION ===")

# Add unique IDs to DataFrame
df_with_ids = df_employees.withColumn("unique_id", monotonically_increasing_id(
df_with_ids.select("unique_id", "first_name", "last_name").show()

# Use for creating surrogate keys
df_surrogate_key = df_employees.withColumn(
    "employee_key",
    monotonically_increasing_id()
).withColumn(
    "row_hash",
    sha2(concat_ws("|", col("first_name"), col("last_name"), col("email")), 256
)

df_surrogate_key.select("employee_key", "first_name", "last_name", "row_hash").

# Real-world application: Data lineage tracking
df_with_lineage = df_employees.withColumn("record_id", monotonically_increasing
    .withColumn("created_timestamp", current_timestamp()) \
    .withColumn("source_system", lit("HR_SYSTEM")) \
    .withColumn("batch_id", lit("BATCH_2024_001"))

print("Data with lineage tracking:")
df_with_lineage.select("record_id", "first_name", "created_timestamp", "source_
```

```
# Use in data deduplication process
df_dedupe_prep = df_employees.withColumn("row_id", monotonically_increasing_id(
    .withColumn("row_number",
                row_number().over(Window.partitionBy("first_name", "last_name").

# Keep only the first occurrence of each name combination
df_deduped = df_dedupe_prep.filter(col("row_number") == 1)
print(f"Original records: {df_employees.count()}")
print(f"After deduplication: {df_deduped.count()}")
```

46. 📁 input_file_name() — The File Tracker

Track the source file of each record (useful when reading multiple files):

```
print("=== FILE SOURCE TRACKING ===")

# This function is most useful when reading from multiple files
# For demonstration, we'll simulate this scenario

# First, let's save our data to multiple files
df_employees.filter(col("department") == "Engineering").write.mode("overwrite")
df_employees.filter(col("department") == "Marketing").write.mode("overwrite").p
df_employees.filter(col("department") == "Sales").write.mode("overwrite").parqu

# Read from multiple files and track source
df_multi_source = spark.read.parquet("engineering_employees", "marketing_employ

# Add source file information
df_with_source = df_multi_source.withColumn("source_file", input_file_name()) \
    .withColumn("file_name_only",
                regexp_extract(input_file_name(), "([^/]+)/?$", 1))

print("Data with source file tracking:")
df_with_source.select("first_name", "department", "file_name_only").show(trunca

# Analyze data distribution by source
source_analysis = df_with_source.groupBy("file_name_only").agg(
    count("*").alias("record_count"),
    countDistinct("department").alias("unique_departments"),
    collect_set("department").alias("departments")
)

print("Source file analysis:")
source_analysis.show(truncate=False)

# Real-world application: Data quality by source
quality_by_source = df_with_source.groupBy("file_name_only").agg(
```

```
    count("*").alias("total_records"),
    sum(when(col("email").isNull(), 1).otherwise(0)).alias("null_emails"),
    sum(when(col("salary") <= 0, 1).otherwise(0)).alias("invalid_salaries"),
    avg("salary").alias("avg_salary")
).withColumn(
    "data_quality_score",
    (col("total_records") - col("null_emails") - col("invalid_salaries")) / col
)

print("Data quality analysis by source:")
quality_by_source.show()

# Clean up temporary files
import shutil
import os
for dir_name in ["engineering_employees", "marketing_employees", "sales_employe
    if os.path.exists(dir_name):
        shutil.rmtree(dir_name)
```

🎉 GRAND FINALE: Putting It All Together!

Now let's create a comprehensive real-world data pipeline that uses multiple functions we've learned:

```
def comprehensive_data_pipeline():
    """
    A complete data engineering pipeline demonstrating the power of PySpark
    """
    print("🚀 EXECUTING COMPREHENSIVE DATA PIPELINE 🚀")
    print("=" * 60)

    # Step 1: Data Ingestion and Quality Assessment
    print("📥 STEP 1: DATA INGESTION & QUALITY ASSESSMENT")

    df_raw = df_employees.withColumn("ingestion_timestamp", current_timestamp()
        .withColumn("record_id", monotonically_increasing_id())

    # Data quality metrics
    total_records = df_raw.count()
    quality_metrics = df_raw.agg(
        count("*").alias("total_records"),
        sum(when(col("email").isNull(), 1).otherwise(0)).alias("null_emails"),
        sum(when(col("salary") <= 0, 1).otherwise(0)).alias("invalid_salaries")
        countDistinct("first_name", "last_name").alias("unique_names")
    ).collect()[0]

    print(f"✅ Ingested {quality_metrics['total_records']} records")
    print(f"⚠️  Data Quality Issues: {quality_metrics['null_emails']} null ema
```

```
# Step 2: Data Cleaning and Standardization
print("\n🧹 STEP 2: DATA CLEANING & STANDARDIZATION")

df_cleaned = df_raw.fillna({"email": "no-email@company.com"}) \
    .withColumn("full_name", concat(col("first_name"), lit(" "), col("last_
    .withColumn("hire_date_formatted", to_date(col("hire_date"), "yyyy-MM-d
    .withColumn("tenure_years",
                round(datediff(current_date(), to_date(col("hire_date"), "yy
    .withColumn("skill_count", size(col("skills"))) \
    .withColumn("email_domain", regexp_extract(col("email"), "@(.+)", 1)) \
    .cache()  # Cache for multiple operations

print("✅ Data cleaned and standardized")

# Step 3: Feature Engineering
print("\n⚙️ STEP 3: FEATURE ENGINEERING")

# Create salary bands, seniority levels, and performance indicators
df_featured = df_cleaned.withColumn(
    "salary_band",
    when(col("salary") >= 90000, "Executive")
    .when(col("salary") >= 70000, "Senior")
    .when(col("salary") >= 50000, "Mid-level")
    .otherwise("Entry-level")
).withColumn(
    "seniority_level",
    when(col("tenure_years") >= 5, "Veteran")
    .when(col("tenure_years") >= 2, "Experienced")
    .otherwise("New")
).withColumn(
    "skill_diversity",
    when(col("skill_count") >= 4, "Highly Diverse")
    .when(col("skill_count") >= 2, "Moderately Diverse")
    .otherwise("Specialized")
).withColumn(
    "performance_score",
    (col("salary") / 1000 + col("skill_count") * 5 + col("tenure_years") *
)

# Add window functions for ranking
dept_window = Window.partitionBy("department").orderBy(col("performance_scc
overall_window = Window.orderBy(col("performance_score").desc())

df_ranked = df_featured.withColumn(
    "dept_performance_rank", row_number().over(dept_window)
).withColumn(
    "overall_performance_rank", row_number().over(overall_window)
).withColumn(
    "salary_percentile", ntile(100).over(Window.orderBy("salary"))
)

print("✅ Features engineered and rankings calculated")
```

```
# Step 4: Advanced Analytics
print("\n📊 STEP 4: ADVANCED ANALYTICS")

# Department analytics
dept_analytics = df_ranked.groupBy("department").agg(
    count("*").alias("headcount"),
    avg("salary").alias("avg_salary"),
    avg("performance_score").alias("avg_performance"),
    avg("tenure_years").alias("avg_tenure"),
    collect_set(explode(col("skills"))).alias("unique_skills"),
    sum("salary").alias("total_payroll")
).withColumn(
    "payroll_percentage",
    round(col("total_payroll") / sum("total_payroll").over(Window.partitior
)

print("Department Analytics:")
dept_analytics.show()

# Skills analysis with market value
skills_value = df_ranked.select(explode(col("skills")).alias("skill"), "sal
    .groupBy("skill").agg(
        count("*").alias("frequency"),
        avg("salary").alias("avg_salary_with_skill"),
        countDistinct("department").alias("departments_using"),
        collect_set("department").alias("dept_list")
    ).orderBy(col("avg_salary_with_skill").desc())

print("Skills Market Value Analysis:")
skills_value.show()

# Step 5: Predictive Insights
print("\n🔮 STEP 5: PREDICTIVE INSIGHTS")

# Identify high-potential employees
high_potential = df_ranked.filter(
    (col("performance_score") > df_ranked.agg(avg("performance_score")).col
    (col("tenure_years") < 3) &
    (col("skill_count") >= 3)
).select("full_name", "department", "performance_score", "salary", "skill_c

print("High-Potential Employees (High performance, low tenure, diverse skil
high_potential.show()

# Retention risk analysis
retention_risk = df_ranked.withColumn(
    "retention_risk",
    when(
        (col("salary_percentile") < 25) & (col("tenure_years") > 2), "High
    ).when(
        (col("dept_performance_rank") > 3) & (col("skill_count") >= 3), "Me
    ).otherwise("Low Risk")
```

```
    ).filter(col("retention_risk") != "Low Risk")

    print("Retention Risk Analysis:")
    retention_risk.select("full_name", "department", "salary_percentile", "rete

    # Step 6: Business Intelligence Dashboard Data
    print("\n📈 STEP 6: BUSINESS INTELLIGENCE SUMMARY")

    # Executive summary
    executive_summary = df_ranked.agg(
        count("*").alias("total_employees"),
        avg("salary").alias("avg_company_salary"),
        sum("salary").alias("total_payroll"),
        avg("performance_score").alias("avg_performance"),
        countDistinct("department").alias("departments"),
        avg("tenure_years").alias("avg_tenure"),
        countDistinct(explode(col("skills"))).alias("unique_skills_company")
    )

    print("Executive Summary:")
    executive_summary.show()

    # Trend analysis
    hiring_trends = df_ranked.withColumn("hire_year", year(col("hire_date_forma
        .groupBy("hire_year").agg(
            count("*").alias("hires"),
            avg("salary").alias("avg_starting_salary"),
            avg("skill_count").alias("avg_skills_at_hire")
        ).orderBy("hire_year")

    print("Hiring Trends by Year:")
    hiring_trends.show()

    # Step 7: Data Export and Cleanup
    print("\n💾 STEP 7: DATA EXPORT & CLEANUP")

    # In a real scenario, you would save to various formats
    # df_ranked.write.mode("overwrite").parquet("processed_employee_data")
    # dept_analytics.write.mode("overwrite").json("department_analytics")

    # Unpersist cached DataFrames
    df_cleaned.unpersist()

    print("✅ Pipeline completed successfully!")
    print("📊 Data processed, analyzed, and ready for business consumption")

    return df_ranked, dept_analytics, skills_value

# Execute the comprehensive pipeline
final_employee_data, department_insights, skills_insights = comprehensive_data_
```

🎊 Conclusion: Your PySpark Mastery Journey Complete!

Congratulations, Data Warrior! 🏆 You've just completed an epic journey through all 46 essential PySpark functions! This isn't just a tutorial — it's your comprehensive toolkit for conquering the world of big data.

🌟 What You've Mastered:

✅ Data Selection & Manipulation- *select(), withColumn(), filter()*

✅ Data Quality & Cleaning — **dropDuplicates(), fillna(), isNull()**

✅ Advanced Aggregations -**groupBy(), collect_list(), pivot()**

✅ Window Functions — **row_number(), lag(), ntile()**

✅ Complex Data Types -**explode(), struct(), array()**

✅ Performance Optimization -**cache(), broadcast(), repartition()**

✅ String & Date Operations -**regexp_extract(), date_format()**

✅ Statistical Analysis — **corr(), describe(), stddev()**

✅ Streaming Processing -**foreachBatch(), foreach()**

✅ SQL Integration -**createOrReplaceTempView(), spark.sql()**

🚀 Your Next Steps to Data Engineering Excellence:

1. **Practice with Real Data** 📊 — Apply these functions to your own datasets
2. **Build End-to-End Pipelines** 🔄 — Create complete data processing workflows
3. **Optimize for Performance** ⚡ — Master partitioning, caching, and broadcast joins
4. **Explore MLlib** 🤖 — Dive into machine learning with PySpark
5. **Learn Streaming** 🌊 — Master real-time data processing
6. Join the Community 👥 — Connect with other PySpark practitioners

💡 Key Principles to Remember:

🎯 **Think Distributed** — Always consider how your operations will scale

🔧 **Optimize Early** — Use caching and partitioning strategically

🧹 **Clean First** — Data quality is the foundation of good analytics
📊 **Measure Everything** — Use window functions for advanced analytics
🔄 **Chain Operations** — Build efficient transformation pipelines

🌈 The Power You Now Possess:

With these 46 functions, you can:

- Process petabytes of data across clusters 🌐
- Build real-time analytics systems 📡
- Create sophisticated ML pipelines 🤖
- Optimize performance for massive scale ⚡
- Bridge SQL and Python seamlessly 🌉

🎉 Final Words of Wisdom:

Remember, mastering PySpark is like learning to conduct a symphony orchestra. Each function is an instrument, and when you combine them skillfully, you create beautiful data harmonies that can transform businesses and drive insights! 🎶

The journey doesn't end here — it's just the beginning. The world of big data is vast and exciting, and you now have the tools to explore every corner of it. Keep experimenting, keep learning, and most importantly, keep having fun with your data adventures!

Happy Sparking, Data Maestro! ⚡ 🌀 ✨

"In the world of big data, PySpark is not just a tool — it's your superpower. Use it wisely, and there's no limit to what you can achieve!"

🔥 Now go forth and transform the world, one DataFrame at a time! 🔥

AI | Spark | Big Data | Technology | Programming

Follow

Written by Mayurkumar Surani

1.8K followers · 828 following

Data Engineer | Data Scientist | Machine Learner | Digital Citizen